JAVA CONSTRUCTORS

Complete Interview-Ready Notes | Beginner to Advanced

1. WHAT IS A CONSTRUCTOR?

A Constructor is a special method in Java that is automatically called when an object is created using the new keyword. It is used to initialize the object's state (variables).

Simple Real-World Analogy:

Think of a Constructor like a 'Registration Form' when you join a new school. The moment you join (object is created), you fill in your name, age, class (initial values). That's exactly what a constructor does — it sets up the object right from the start.

Syntax:

```
class ClassName {
    ClassName() {           // Constructor - same name as class
        // initialization code
    }
}
```

Key Characteristics of a Constructor:

- ✓ Name must be EXACTLY same as the class name
- ✓ No return type — not even void
- ✓ Called automatically when object is created using 'new'
- ✓ Can have parameters (or no parameters)
- ✓ Cannot be static, abstract, or final
- ✓ Can be overloaded (multiple constructors with different parameters)

> 💡 **TIP:** Constructor name = Class name. This is how Java identifies it as a constructor!

2. WHY USE CONSTRUCTORS?

Without constructors, every time you create an object, you'd have to manually set every variable using setter methods. Constructors automate initialization and ensure the object is in a valid state from the moment it's created.

Without Constructor (Bad Practice):

```
class Student {
    String name;
    int age;
```

```
}

Student s = new Student();
s.name = "Rahul";     // manually setting every time
s.age = 20;            // error-prone and repetitive
```

With Constructor (Best Practice):

```
class Student {
    String name;
    int age;

    Student(String name, int age) {   // Constructor
        this.name = name;
        this.age  = age;
    }
}

Student s = new Student("Rahul", 20); // clean & safe!
```

> ⚠ **NOTE:** If you don't define any constructor, Java automatically provides a Default Constructor with no arguments. But once you define your own, Java removes the default one.

3. TYPES OF CONSTRUCTORS

Type 1 — Default Constructor (No-Arg Constructor)

A constructor with NO parameters. Java provides it automatically if you don't write any constructor. It initializes variables to default values (0, null, false).

```
class Bike {
    String brand;
    int speed;

    Bike() {                    // Default Constructor
        brand = "Generic";
        speed = 0;
        System.out.println("Bike object created!");
    }
}

Bike b = new Bike();   // calls Default Constructor
// Output: Bike object created!
```

Type 2 — Parameterized Constructor

A constructor that accepts arguments to initialize an object with specific values. Most commonly used in real projects.

```
class Employee {
```

```
    String name;
    int    salary;
    String dept;

    Employee(String name, int salary, String dept) {
        this.name   = name;
        this.salary = salary;
        this.dept   = dept;
    }

    void display() {
        System.out.println(name + " | " + dept + " | Rs." + salary);
    }
}

Employee e1 = new Employee("Priya", 50000, "IT");
Employee e2 = new Employee("Amit", 60000, "HR");
e1.display();  // Priya | IT | Rs.50000
e2.display();  // Amit  | HR | Rs.60000
```

Type 3 — Copy Constructor

A constructor that creates a new object by copying values from an existing object of the same class. Java doesn't provide this automatically — you write it manually.

```
class Rectangle {
    int length, width;

    Rectangle(int length, int width) {   // Parameterized
        this.length = length;
        this.width  = width;
    }

    Rectangle(Rectangle r) {             // Copy Constructor
        this.length = r.length;
        this.width  = r.width;
    }
}

Rectangle r1 = new Rectangle(10, 5);
Rectangle r2 = new Rectangle(r1);  // copy of r1
System.out.println(r2.length);     // 10
```

> 💡 **TIP:** Copy Constructor creates a Deep Copy — both r1 and r2 are independent objects!

4. CONSTRUCTOR OVERLOADING

Having multiple constructors in a class with different parameter lists. Java decides which one to call based on the arguments passed.

```
class BankAccount {
    String owner;
    double balance;
```

```
    String type;

    BankAccount() {
        owner   = "Unknown";
        balance = 0.0;
        type    = "Savings";
    }

    BankAccount(String owner) {
        this.owner = owner;
        balance    = 0.0;
        type       = "Savings";
    }

    BankAccount(String owner, double balance, String type) {
        this.owner   = owner;
        this.balance = balance;
        this.type    = type;
    }

    void show() {
        System.out.println(owner+" |"+type+" | Rs."+balance);
    }
}

BankAccount a1 = new BankAccount();
BankAccount a2 = new BankAccount("Sneha");
BankAccount a3 = new BankAccount("Rohan", 25000.0, "Current");
a1.show();  // Unknown | Savings | Rs.0.0
a2.show();  // Sneha   | Savings | Rs.0.0
a3.show();  // Rohan   | Current | Rs.25000.0
```

5. this() — CONSTRUCTOR CHAINING

You can call one constructor from another in the same class using this(). This avoids code duplication. The this() call must be the FIRST statement in the constructor.

```
class Pizza {
    String size;
    String crust;
    String topping;

    Pizza() {
        this("Medium", "Thin", "Cheese"); // calls 3rd constructor
    }

    Pizza(String size) {
        this(size, "Thin", "Cheese");      // calls 3rd constructor
    }

    Pizza(String size, String crust, String topping) {
        this.size    = size;
        this.crust   = crust;
        this.topping = topping;
    }
```

```
    void show() {
        System.out.println(size+" | "+crust+" | "+topping);
    }
}

Pizza p1 = new Pizza();
Pizza p2 = new Pizza("Large");
p1.show();  // Medium | Thin | Cheese
p2.show();  // Large  | Thin | Cheese
```

> ⚠ **NOTE:** this() must always be the FIRST line in the constructor — you cannot put any code before it!

6. super() — PARENT CLASS CONSTRUCTOR

In Inheritance, the child class constructor calls the parent class constructor using super(). If you don't write super(), Java automatically inserts super() as the first line.

```
class Animal {
    String name;

    Animal(String name) {
        this.name = name;
        System.out.println("Animal created: " + name);
    }
}

class Dog extends Animal {
    String breed;

    Dog(String name, String breed) {
        super(name);              // calls Animal's constructor FIRST
        this.breed = breed;
        System.out.println("Dog breed: " + breed);
    }
}

Dog d = new Dog("Bruno", "Labrador");
// Output:
// Animal created: Bruno
// Dog breed: Labrador
```

> 💡 **TIP:** Parent constructor always runs before child constructor. super() must also be the first line!

7. COMPARISON TABLE

Feature	Constructor	Method
Name	Same as class name	Any valid name
Return Type	No return type (not even void)	Must have return type
Called by	Automatically with 'new'	Explicitly by programmer
Purpose	Initialize object state	Define object behavior
Inheritance	Not inherited	Inherited
Overloading	Yes, allowed	Yes, allowed
this() / super()	Yes, can use	Cannot use this() / super()

8. CONSTRUCTOR EXECUTION ORDER

When you create an object, Java follows this exact order:

- ✓ Static variables and static blocks (only once, when class loads)
- ✓ Instance variables are initialized to default values
- ✓ Instance initializer blocks run (top to bottom)
- ✓ Constructor body runs

```
class Demo {
    static int x = 10;                // 1st: static var
    int y = 20;                       // 3rd: instance var

    static { System.out.println("Static block: x=" + x); }  // 2nd
    { System.out.println("Instance block: y=" + y); }       // 4th

    Demo() { System.out.println("Constructor called"); }    // 5th
}

// Output when Demo d = new Demo() :
// Static block: x=10
// Instance block: y=20
// Constructor called
```

9. PRIVATE CONSTRUCTOR (Singleton Pattern)

A private constructor prevents other classes from creating objects. This is used in the Singleton Design Pattern — ensuring only ONE object of a class exists.

```
class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {            // private!
        System.out.println("DB Connected!");
    }
```

```
    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}

DatabaseConnection db1 = DatabaseConnection.getInstance();
DatabaseConnection db2 = DatabaseConnection.getInstance();
System.out.println(db1 == db2);  // true — same object!
```

> 💡 **TIP:** Private Constructor + Singleton Pattern = Very COMMON interview topic. Memorize this!

10. TOP INTERVIEW QUESTIONS & ANSWERS

Q: Can a constructor return a value?

A: No. Constructors have no return type — not even void. They implicitly return the instance of the class. If you add a return type, Java treats it as a regular method, not a constructor.

Q: What happens if you don't define a constructor?

A: Java automatically provides a Default Constructor (no-arg) that initializes all instance variables to default values — 0 for int, null for String, false for boolean.

Q: Can a constructor be private?

A: Yes! A private constructor restricts object creation from outside the class. It's commonly used in Singleton Pattern, Factory Pattern, and Utility classes.

Q: What is the difference between constructor and method?

A: Constructor: same name as class, no return type, auto-called. Method: any name, must have return type, called explicitly. Constructor initializes; method defines behavior.

Q: Can we use 'this' and 'super' both in one constructor?

A: No. You can only use ONE of them, and it must be the FIRST statement. Using both would require two first-line calls which is impossible.

Q: Is constructor inherited in Java?

A: No. Constructors are NOT inherited. A subclass must define its own constructor. However, it CAN call the parent constructor using super().

Q: Can a constructor call another constructor in same class?

A: Yes, using this(). This is called Constructor Chaining. The this() call must be the very first statement in the constructor body.

Q: Can we overload constructors?

A: Yes! Constructor Overloading means having multiple constructors in the same class with different parameter lists. Java picks the right one based on arguments passed.

Q: What is a Copy Constructor?

A: A constructor that takes an object of the same class as argument and creates a new object by copying its field values. Java does NOT provide it automatically — you write it.

Q: What are default values given by Default Constructor?

A: int/long/short/byte → 0, float/double → 0.0, boolean → false, char → '\u0000', Object references (String, etc.) → null.

11. QUICK CHEAT SHEET

Constructor Type	Syntax	When to Use
Default	ClassName() { }	When you want default initial state
Parameterized	ClassName(args) { }	When you want custom initial values
Copy	ClassName(ClassName obj) { }	When you want a clone of object
Chaining (this)	ClassName() { this(args); }	Avoid code duplication
Super call	Child() { super(args); }	Initialize parent class fields
Private	private ClassName() { }	Singleton / Factory Pattern

12. COMMON MISTAKES TO AVOID

⚠ **NOTE:** Adding void or any return type to constructor — it becomes a method, not a constructor!

⚠ **NOTE:** Naming constructor differently from class name — Java won't recognize it.

⚠ **NOTE:** Calling this() or super() after another statement — must always be first line.

⚠ **NOTE:** Trying to call a constructor like a normal method — new keyword is mandatory.

⚠ **NOTE:** Using both this() and super() in the same constructor — only one is allowed.

All the best for your Interview! Keep Coding